1- Clarify the differences between Re-engineering and Refactoring

Re-engineering:

When: Takes place when maintenance costs are increasing.

What: Restructuring or rewriting a legacy system.

Goal: To make the system easier to maintain.

Refactoring:

When: A continuous process during development and evolution.

What: Modifying code to improve structure and reduce complexity.

Goal: Preventative maintenance to avoid code degradation.

2- Define the meaning of software measurements, metrics, assessments and explain metric types.

Software Measurement: means giving a number to a feature of the software or the process.

Assessments: Using numbers (measurements) to judge the quality of the software.

Software Metric: A measurement for a software system, process, or documents.

Metric Types:

Control metrics: Measurements about the process (how we work), like the time spent fixing errors.

Predictor metrics: Measurements of the product (software) used to predict its quality, like Module complexity and identifier length

Dynamic metrics: Collected during execution (assess efficiency/reliability)

Static metrics: Collected from system representations (assess complexity/maintainability).

3-Illustrate/Discuss the Configuration Management activities.

Change Management: Tracking requests and deciding which changes to implement.

Version Management: Managing different versions to prevent conflicts between developers.

System Building: Assembling and compiling components into a working (executable) system.

Release Management: Preparing and tracking software versions sent to customers.

4-Clarify how to minimize the recompilation?

Tools avoid unnecessary recompilation by checking a **Unique Signature**:

Modification Timestamps:

The signature is the **date/time** of the file.

Rule: If the **Source file** is newer than the **Object file**, recompile.

Source Code Checksums:

A unique **number** calculated from the code text.

Rule: If the code changes (even by one character), the checksum changes, and it recompiles.

5-Identify three situations that lead to software maintenance and mention why software maintenance is difficult? Why is it necessary?

Maintenance:

Repairing faults: Fixing bugs or errors.

Adapting environment: Changing the software to work on new operating systems or hardware.

Modifying functionality: Adding or changing features to meet new requirements.

Difficult:

High Costs: It often costs more than the original development.

Complexity: It is affected by both technical and non-technical factors.

Structure Degradation: Continuous changes damage the software structure, making it harder to maintain.

Software Aging: Older systems have very high support costs.

Necessary:

Stability & Responsibility: It ensures team stability and fulfills legal duties.

Motivation: To support developers to design for future changes .

Knowledge Management: To help new staff understand old and complex code

6-What is process improvement? Mention its approaches to improvement.

Definition: Changing curreningt processes to improve **quality**, and reduce **costs** or **time**.

Approaches:

1. **Process Maturity**: Focuses on better **management** and following standards like **CMMI**.
2. **Agile Approach**: Focuses on **speed**, **iterative development**, and reducing extra work (overheads).

7-Illustrate what is meant by Quality Management process and discuss its activities.

Meaning: Ensuring that the software reaches the required level of quality.

Activities:

1. **Quality Assurance (QA)**: Setting general quality standards and procedures for the organization.
2. **Quality Planning (QP)**: Choosing the best standards and procedures for a specific project.
3. **Quality Control (QC)**: Checking that the development team is following the chosen standards.

8-Discuss briefly the Handover problems.

Handover problems occur when the development and evolution teams use different methodologies:

- **Agile to Plan-based**: The development team uses **Agile**, but the evolution team prefers a **Plan-based** approach.
- **Plan-based to Agile**: The development team uses **Plan-based**, but the evolution team prefers **Agile** methods.

9-Process Analysis Objectives and Techniques

Objectives (What we want to achieve):

- **Understand Activities**: Identify tasks and how they connect.
- **Link Measurements**: Relate process tasks to metrics/data.
- **Compare Processes**: Relate the current process to ideal models.

Techniques (How we do it):

- **Published Models**: Use existing standards as a starting point.
- **Interviews**: Ask participants specific questions to get info.
- **Ethnography**: Observe work directly to understand it deeply.

10-Differentiate among the following system situations: Evolution, Servicing, Phase-out.

- **Evolution**: The system grows by adding **new requirements** and features.

- **Servicing**: Only **bug fixes** are made; **no new features** are added.

- **Phase-out**: The system is still used, but **no more changes** are made at all.

11- Illustrate the Process improvement stages. The cycle involves.

- **Process Measurement**: Measure the current work to set a **baseline** (starting point).

- **Process Analysis**: Find **weaknesses** and **bottlenecks** (problems) in the current process.

- **Process Change**: **Implement** (apply) the changes to solve the identified problems.

12-Speak about documentation standards and state its types.

Documents are the **tangible proof** of the software.

Process Standards: Rules for **how** to create, check, and update documents.

Document Standards: Rules for the **content, structure, and look** of the document.

Interchange Standards: Rules to make files **compatible** across different systems (like PDF or XML).

13-You receive a change request for software that you maintain. Explain the process of handling and implementing the request.

1. **Request**: Fill out a **Change Form**.
2. **Analyze**: Check if the request is **valid** (correct).
3. **Assess**: Estimate the **cost** and effort needed.
4. **Submit**: Send the request to the **CCB** (Control Board).
5. **Decision**:
    - If **accepted**: Update code and release a **new version**.
    - If **rejected**: Inform the requester.

14-The GQM paradigm is used in process improvement to help answer three critical questions. Explain the GQM paradigm and what are these three questions.

QM Paradigm Summary

- **Goals**: What the organization wants to **achieve**.
- **Questions**: Things we need to **know** to reach the goals.
- **Metrics**: The **data** and measurements used to answer the questions.

The Three Critical Questions

1. **Why** are we improving the process?
2. **What information** is needed to identify improvements?
3. **What measurements** are required to get this information?

15-Explain the cases of software change is inevitable?

New requirements emerge when the software is used.

The business environment changes.

Errors must be repaired.

New computers and equipment are added to the system.

Performance or reliability must be improved.

16-State 4 questions to ensure Software fitness for purpose.

1. **Rules**: Did we follow the **coding and documentation rules**?
2. **Testing**: Was the software **fully tested** for errors?
3. **Trust**: Is the software **stable and safe** enough to use?
4. **Speed**: Is the **performance (speed)** good enough for the users?

17-Discuss what is Configuration management and why?

Definition: The rules and tools used to **manage changes** in a software system.

Why?: Because software changes a lot, and we need to **track different versions** so we don't lose any information.